

Claude Code Implementation Handover — 2026-05-15

Project: CJ Panganiban Conversation App (Supervaise FLP Project 2)

Repo: <https://github.com/jeanettepao-ux/Supervaise-FLP-Project-2>

Last commit: 839a4d2 on main, clean working tree

This document: implementation truth as of today. Pair with the strategic/decision handover (separate document) for full context.

1. TL;DR

The project runs end-to-end today. The text-only smoke test (`python cj_chat.py --text "What is the rule of law?"`) passes — full output captured in §4. The Streamlit chat dashboard (`streamlit run dashboard.py`) serves HTTP 200 and has been used interactively. Both the CLI and the dashboard share the same pipeline: faster-whisper STT -> Claude Haiku router -> Claude Sonnet inference -> Piper TTS. There are **no automated tests** in the repo; verification has been one-off human-driven runs of the 6 sanity-check questions from the build kit. The Tagalog TTS is approximated by hand-crafted phonetic substitutions (stopgap, not native quality); everything else behaves as the build-kit spec intended.

2. Repo layout

Supervaise-FLP-Project-2/		
■■■ README.md	[implemented]	project intro + quick start
■■■ PROJECT.md	[implemented]	comprehensive project doc (456 lines)
■■■ .gitignore	[implemented]	excludes .env, .venv/, app/voices/*.onnx, app/piper/, app/state/
■■■ docs/		
■ ■■■ handover_claude_code_2026-05-15.md	[this file]	
■■■ app/		the runnable app
■ ■■■ README.md	[implemented]	app run instructions (212 lines)
■ ■■■ cj_chat.py	[working]	CLI + all pipeline functions (616 lines)
■ ■■■ dashboard.py	[working]	Streamlit chat UI (367 lines)
■ ■■■ requirements.txt	[implemented]	
■ ■■■ .env	[implemented; GITIGNORED]	ANTHROPIC_API_KEY + paths
■ ■■■ .env.example	[implemented]	
■ ■■■ .gitignore	[implemented]	app-scoped ignores (state/, voices/*.onnx, piper/)
■ ■■■ artifacts/		corpus artifacts loaded at startup
■ ■ ■■■ voice_card.md	[implemented]	inference system prompt (229 lines)
■ ■ ■■■ router_prompt.md	[implemented]	router system prompt (198 lines)
■ ■ ■■■ topic_map.json	[implemented]	37 canonical topics
■ ■ ■■■ topic_graph.json	[implemented]	37 nodes, 78 edges
■ ■ ■■■ entity_index.json	[implemented]	69 people, 16 cases, laws_treaties
■ ■ ■■■ frameworks.json	[implemented]	10 named frameworks (four_ins, three_es, etc.)
■ ■ ■■■ signature_library.json	[implemented]	~684 normalized signature phrases
■ ■ ■■■ topics/	[implemented]	89 Stage-1 per-doc extractions
■ ■ ■■■ piper/	[installed; GITIGNORED]	Piper Windows binary + dlls (38 MB)
■ ■ ■■■ voices/	[installed; GITIGNORED]	en_US-ryan-high voice (116 MB)
■ ■ ■■■ state/	[runtime; GITIGNORED]	generated TTS waves land here
■ ■ ■■■ .venv/	[GITIGNORED]	Python 3.11 venv (464 MB)
■■■ corpus/		source corpus + pre-processing
■ ■■■ manifest.json		
■ ■■■ build_kit/	[implemented]	voice_card.md, router_prompt.md, cj_chat.py reference, README.md (the spec)
■ ■■■ prompts/	[implemented]	stage_1a_extraction.md, stage_3_design.md
■ ■■■ synthesis_scripts/	[implemented]	taxonomy.py, synthesizer.py, make_summary.py, aliases.py, stage_3_design.md
■ ■■■ analysis/		
■ ■ ■■■ synthesis/	[implemented]	Stage 3 outputs (canonical layer, 7 files)
■ ■ ■■■ topics/	[implemented]	Stage 1 outputs (89 per-doc extractions)
■■■ source_materials/		original published writing (~150K words)
■■■ manifest.json		
■■■ books/book_01_centenary-of-justice/	[implemented]	25 .md files (chapters + appendices)
■■■ columns/	[implemented]	65 .md files (2011-2026)

Everything marked **[implemented]** exists on disk and is wired in.

[working] = exercised by at least one successful run today.

No directory is a stub or empty.

3. What's been built

Component	Status	File(s)	Notes
STT (faster-whisper)	working	<code>app/cj_chat.py:134-144</code>	<code>medium</code> model, int8 quantized, CPU. VAD on (<code>min_silence_duration_ms=500</code>). Verified round-trip on Piper-synthesized wav.
Router (Haiku)	working	<code>app/cj_chat.py:150-183</code>	<code>claude-haiku-4-5-20251001</code> , <code>max_tokens=300</code> , JSON output. Code-fence stripping + topic-ID validation against <code>topic_map.json</code> with <code>rule_of_law</code> fallback. All 6 build-kit sanity questions routed correctly.
Retrieval (build_context)	working	<code>app/cj_chat.py:189-243</code>	Loads 1-3 routed topics from <code>topic_map.json</code> , top-3 raw docs of primary topic from <code>artifacts/topics/</code> , trimmed to essentials. Outputs an XML-tagged context block.
Inference (Sonnet)	working	<code>app/cj_chat.py:249-286</code>	<code>claude-sonnet-4-6</code> , <code>max_tokens=600</code> , <code>voice_card.md</code> as system prompt, confidence-aware grounding note. Pipes response through <code>_strip_stage_directions</code> before return.
TTS (Piper)	working	<code>app/cj_chat.py:413-442</code>	Multi-line stdin, <code>--sentence_silence 0.6</code> , <code>--length_scale 1.05</code> , <code>--quiet</code> . Outputs single concatenated wav.
TTS text prep	working	<code>app/cj_chat.py:325-372</code>	Strips markdown, applies phonetic substitutions for Tagalog/Spanish/French (12 entries), converts long-dashes -> commas, splits on sentence boundaries.

Stage-direction stripping	working	<code>app/cj_chat.py:391-410</code>	Regex <code>^\s**[\^*\n]+*\s*\$</code> removes whole-line italics like <code>*A moment of quiet*</code> ; inline emphasis preserved.
Audio I/O (CLI push-to-talk)	working	<code>app/cj_chat.py:459-514</code>	RMS-based silence detection, ~1.2s trailing silence, 30s max. Streams via <code>sounddevice.InputStream</code> .
Audio I/O (dashboard mic)	working	<code>app/dashboard.py:206-241</code>	<code>st.audio_input</code> widget, browser records, MD5-hash to avoid re-processing on Streamlit rerun, mic-counter key trick to reset widget per turn.
Audio playback	working	<code>app/cj_chat.py:445-453</code>	Cross-platform: <code>winsound</code> on win32, <code>afplay</code> on darwin, <code>aplay</code> on linux. Dashboard uses <code>st.audio(..., autoplay=True)</code> .
Conversation history	working	<code>app/cj_chat.py:600-610</code> , <code>app/dashboard.py:97-99, 254-255</code>	Last 10 turns (20 messages) passed to inference call for multi-turn coherence. Persisted in <code>st.session_state.messages</code> (dashboard) or in-process list (CLI).
Anthropic retry handling	working	<code>app/cj_chat.py:90-95</code>	<code>make_client()</code> returns <code>Anthropic(max_retries=4)</code> — ~15s of internal retry on 5xx/429 before bubbling up.
API error handling (dashboard)	working	<code>app/dashboard.py:286-322</code>	Catches <code>APIStatusError</code> and branches on <code>status_code</code> (529, 429, other); <code>APIConnectionError</code> ; catch-all <code>Exception</code> . Shows <code>st.error</code> banner, persists marker assistant turn so user's question stays in history.
API error handling (CLI)	working	<code>app/cj_chat.py:548-564</code>	try/except around <code>route+generate</code> , prints friendly <code>stderr</code> message.

Sources expander (dashboard)	working	<code>app/dashboard.py:149-187</code>	Topic pills (primary/secondary, color-coded), confidence (color-coded), router reasoning, top-3 doc titles+dates+IDs.
TTS debug expander	working	<code>app/dashboard.py:343-353</code>	Toggle in sidebar. Shows the per-line chunks Piper saw with ■ markers where pauses land.
Voice card (CJ system prompt)	implemented	<code>app/artifacts/voice_card.m229</code>	129 lines. Identity, voice fingerprint, register hierarchy, anchor topics, out-of-corpus policy. From build kit. Unchanged.
Router prompt	implemented	<code>app/artifacts/router_prompt.m198</code>	198 lines. System block parsed from the first ... fence. From build kit. Unchanged.
Topic taxonomy artifacts	implemented	<code>app/artifacts/{topic_map, topics, edges, index, frameworks}</code>	137 topics, 78 edges, 69 people, 16 cases, 10 frameworks, 684 phrases. Generated by <code>corpus/synthesis_scripts/synth</code>
Operator UI (dashboard)	working	<code>app/dashboard.py</code>	Single-page Streamlit chat with mic + text-fallback + history + audio playback + sources + TTS debug. Sidebar toggles for TTS and debug; "Clear conversation" button.
Backend API endpoints	not started	—	No HTTP backend exists. The dashboard is the backend — it embeds the pipeline directly.
Embeddings ingestion	not started	—	None — routing is by topic IDs via a Haiku call, not vector similarity. No embeddings layer exists.
Database schema	not started	—	No DB. Conversation history is in-memory (<code>st.session_state</code> or Python list); artifacts are JSON files on disk.
Robot adapter (Reachy Mini)	not started	—	Out of scope per build-kit README (May 30 demo is conversation-only).

Speech corpus (Pass B)	not started	—	The ~150 CJ speeches called out in the build kit are not in <code>source_materials/</code> . Only book + columns.
Automated tests	not started	—	No <code>tests/</code> directory, no <code>pytest</code> , no fixtures. Verification has been manual smoke tests only.

Status legend per spec:

`not started` = directory or stub only

`in progress` = partial, won't run end-to-end

`runnable` = executes without error, behavior unverified

`working` = executes and produces expected output on ≥1 case

`tested` = automated tests pass

4. What runs right now

Text-only smoke test (no audio, no mic required)

```
cd "D:\Reachy Mini Project\App 2\app"
.venv\Scripts\python.exe cj_chat.py --text "What is the rule of law?"
```

Actual output captured today (2026-05-15):

```
Loading artifacts...
✓ 37 topics loaded

■ You: What is the rule of law?

■ Routing...
  primary: rule_of_law
  secondary: ['liberty_and_prosperity']
  confidence: high
■ Thinking...

■■ CJ: The rule of law is the bedrock of any decent and ordered
society – the supreme principle that law, not force, not the whims of
the powerful, governs us all. In my humble opinion, it is the single
most important organizing concept in our public life.

What does it mean in practice? It means that no one – no president,
no general, no corporation, no foreign power – stands above the law.
It means that rights are protected, not merely promised. And it means
that when any branch of government commits grave abuse of discretion,
the courts must step in – not because judges are superior to the
executive or the legislature, but because, as I wrote in *Obra
Maestra*, the Constitution must be upheld as truly paramount.

But the rule of law cannot stand alone. Liberty without prosperity is
an empty promise – freedom and food, justice and jobs, ethics and
economics must always go together. These twin beacons must always go
together; one is useless without the other.

The opposite of the rule of law is the rule of force – and we see it
today wherever might masquerades as right. *Au contraire* – right
must always be mightier than might. That is the eternal, universal
truth at the heart of this principle.

Mabuhay!
```

Pipeline observable in output: artifacts loaded (37 topics), router returned `rule_of_law` (high) + `liberty_and_prosperity`, response hits every CJ signature marker (twin beacons, *Au contraire*, *Obra Maestra*, chiastic doublets, "In my humble opinion", `Mabuhay!`).

Full voice dashboard

```
cd "D:\Reachy Mini Project\App 2\app"
.venv\Scripts\streamlit run dashboard.py
```

Streamlit listens on `http://localhost:8501` (HTTP 200 verified). First load triggers `@st.cache_resource` for artifacts (~1s), faster-whisper medium model (~12s cold), and Anthropic client (~0.2s). Cold first turn ~25s total; warm subsequent turns ~15-20s.

Voice CLI

```
.venv\Scripts\python.exe cj_chat.py
```

Push-to-talk loop. Not executed end-to-end today, but `record_until_silence` and the surrounding code path were exercised during the build (commit `b7072ef` and prior). No regressions since.

5. Implementation decisions made during build

What	Why	Where in code	Reversible?
Python 3.11 (not 3.12 or 3.14)	faster-whisper / ctranslate2 wheels most stable on 3.11; user's system has 3.11 already installed; 3.14 too new for some deps.	<code>app/.venv/</code> (built with <code>py -3.11 -m venv</code>); README documents.	Yes — rebuild venv with another version.
<code>python-dotenv</code> with <code>load_dotenv(override=True)</code>	User's shell environment had an empty <code>ANTHROPIC_API_KEY</code> shadowing the <code>.env</code> . Default <code>override=False</code> silently kept the empty value.	<code>app/cj_chat.py:47-50</code>	Trivial — change <code>override=True</code> flag.
<code>Anthropic(max_retries=4)</code>	Default SDK retry is 2 (~3s total backoff). User hit a 529 OverloadedError that survived default retries. 4 retries ≈ 15s window — covers most transient overloads without making the user wait too long.	<code>app/cj_chat.py:90-95</code>	Trivial — <code>ANTHROPIC_MAX_RETRIES</code> constant.
Branch on <code>status_code</code> instead of importing <code>OverloadedError</code>	<code>anthropic.OverloadedError</code> lives in private <code>anthropic._exceptions</code> in v0.102.0 — not at top-level namespace. Importing from private modules is fragile across versions.	<code>app/dashboard.py:286-304</code>	Could switch to <code>except OverloadedError</code> if/when SDK exposes it publicly.

Streamlit <code>st.audio_input</code> (browser-side recording)	Originally I built a separate-process two-terminal design (CLI for audio + read-only dashboard polling a state file). User asked for the dashboard to BE the input UI — <code>st.audio_input</code> is Streamlit's built-in browser-mic widget, no extra deps, no native code.	<code>app/dashboard.py:206-212</code>	Yes — could swap back to file-polling design; old design is preserved in commit <code>8311558</code> history.
<code>mic_counter</code> key trick to reset the audio widget	<code>st.audio_input</code> caches its recording across Streamlit reruns. Without resetting the widget key, the same recording would re-submit on every rerun. Bumping <code>f"mic_{counter}"</code> forces Streamlit to create a fresh widget.	<code>app/dashboard.py:102-103, 211, 241</code>	Yes — could use MD5-hash-only approach if we drop the widget reset.
MD5-hash of audio bytes for "already processed?" check	Belt-and-suspenders alongside the <code>mic_counter</code> trick. If the user re-uses an audio file (unlikely but possible), the hash dedupes the request.	<code>app/dashboard.py:222-224</code>	Yes.
TTS pipeline: per-sentence multi-line stdin + <code>--sentence_silence 0.6</code>	Piper concatenates multi-line stdin into one <code>--output_file</code> wav with <code>--sentence_silence</code> between each line. Gives explicit, controllable inter-sentence breath; verified by silent-gap analysis of resulting wav.	<code>app/cj_chat.py:413-442</code>	Reversible — could pass full text as one line and accept Piper's default inter-sentence behavior.
Em-dash -> comma substitution in TTS path (not chunk splitting)	First attempt was splitting on em-dashes too (commit <code>6788eee</code>). User reported "no pause on em-dashes" — turned out to be a Streamlit stale-import issue, but they suggested the simpler text-substitution approach. Confirmed via silent-gap analysis that Piper's native comma pause is actually 150-300ms (occasionally 600ms for clause-boundary commas), strong enough for the dashes. Simpler code, same audible result.	<code>app/cj_chat.py:349-358</code>	Reversible — old chunking code is in commit <code>6788eee</code> history.

TTS_FOREIGN_SUBSTITUTIONS list (Tagalog/Spanish/French phonetic spellings)	Piper's en_US-ryan-high voice is American-English-only. Hand-crafted phonetic spellings ("Mah-RAH-ming sah-LAH-maht poh") give it stress + vowel hints. Stopgap, not native quality. User explicitly chose this over OpenAI TTS swap.	<code>app/cj_chat.py:307-322</code>	Easy to extend (add tuples) or remove (delete list).
Stage-direction stripping: whole-line <code>*...*</code> only	Claude occasionally prefixes responses with <code>*A moment of quiet*</code> style narration. Stripping inline italics (<code>*Au contraire*</code>, <code>*A Centenary of Justice*</code>) would lose emphasis. Regex <code>^\s*\^[^\n]+\s*\$</code> only matches lines that are entirely wrapped — internal asterisks save inline emphasis.	<code>app/cj_chat.py:391-410</code>	Easy — adjust regex.
Streamlit <code>@st.cache_resource</code> for artifacts, whisper, anthropic client	Streamlit reruns the script top-to-bottom on every interaction. Without caching, each rerun would re-load 37 topics + the whisper model + a new Anthropic client. Caching by resource (not data) keeps them as singletons.	<code>app/dashboard.py:73-88</code>	Trivial.
<code>state/tts/</code> for generated wavs	Streamlit's <code>st.audio(path)</code> needs the wav to persist for the lifetime of the page. Tempfiles get cleaned too eagerly; persistent dir under app root works. Gitignored.	<code>app/dashboard.py:91-94, .gitignore</code>	Easy — switch to <code>st.audio(bytes)</code> if disk persistence becomes an issue.
All audio dirs and venvs gitignored, not committed	Voice model is 116 MB, Piper binary is 38 MB, .venv is 464 MB. Committing these blows up the repo and isn't useful — each user installs locally. README and PROJECT.md document where to get them.	<code>app/.gitignore</code>, root <code>.gitignore</code>	Reversible (would not recommend).
<code>.env</code> loaded from <code>Path(__file__).parent / ".env"</code> (not CWD)	Streamlit reruns from various working dirs; CWD-relative <code>.env</code> loading failed in some cases. Resolving relative to <code>cj_chat.py</code> is stable.	<code>app/cj_chat.py:48</code>	Trivial.

Windows stdout <code>reconfigure(encoding="utf-8")</code>	Default Windows console can't print <code>██████</code> emojis and crashes with <code>UnicodeEncodeError</code> . Stdlib <code>sys.stdout.reconfigure</code> fixes it without external deps.	<code>app/cj_chat.py:53-58</code>	Trivial.
--	---	-----------------------------------	----------

6. Deviations from spec

Item	Type	What the spec says	What's on disk	Why
Two-terminal "CLI + read-only dashboard" design	intentional	The strategy/design doc (per my reading of the build-kit README and the original conversation flow) called for the dashboard to be an audience-facing viewer that mirrored what the CLI did.	The dashboard is now self-contained and drives the full pipeline; the CLI is the secondary path.	User explicitly asked for the dashboard to host the mic input (matching a previous app they'd built). I rebuilt as interactive chat in commit <code>b7072ef</code> . The strategy doc may still reference the old design.
TTS em-dash pause (0.5s split vs comma substitution)	discovered	Build kit doesn't prescribe TTS pause behavior. Earlier commits in this repo (<code>7e582d5</code> , <code>6788eee</code>) implemented em-dash splitting with 0.5s pauses.	Current code substitutes em-dashes with commas; pause is now driven by Piper's native comma phoneme (150-300ms typical).	User asked for the simpler approach in their second TTS feedback turn. The earlier "split on em-dash" code is preserved in git history (<code>6788eee</code>).
Streamlit dashboard added to scope	intentional	The build-kit README explicitly says "Web UI — the reference impl is CLI; a web UI is straightforward but separate work."	Streamlit dashboard exists at <code>app/dashboard.py</code> .	User asked for my recommendation, I suggested a hybrid, user said "Build your recommendation".
Anthropic <code>max_retries=4</code>	discovered	Build-kit reference impl uses bare <code>Anthropic()</code> (default <code>max_retries=2</code>).	<code>make_client()</code> returns <code>Anthropic(max_retries=4)</code>	User hit a 529 OverloadedError that survived default retries.

Whisper VAD <code>min_silence_duration_ms=500</code>	unintentional	Build-kit README troubleshooting says: "If [Whisper hallucinations] still happening, increase <code>vad_parameters={'min_silence_duration_ms': 1000}</code> ".	Current code uses 500.	Carried over from the build_kit reference <code>cj_chat.py</code> without bumping. Hasn't bitten us — but the build kit explicitly recommends 1000 if hallucinations occur. Cheap to change.
Model IDs	discovered	Build-kit uses <code>claude-haiku-4-5-2025-08-01</code> and <code>claude-sonnet-4-6</code> .	Same. Both verified available in current Anthropic SDK v0.102.0.	The naming convention <code>sonnet-4-6</code> (no date suffix) is unusual but the API accepts it. Tested: returns valid responses.
TTS pause length 0.6s (not 0.5s)	intentional	Build kit doesn't prescribe.	<code>TTS_SENTENCE_SILENCE = "0.6"</code> (bumped from 0.5 in commit <code>c30e197</code>).	User feedback that pauses needed to be more obvious. Constant at top of <code>cj_chat.py</code> ; easy to revert.
No automated tests	intentional	Build kit doesn't mandate tests; verification is the 6-question sanity check.	No <code>tests/</code> directory.	Within the demo timeline this is acceptable but worth flagging. See §9.
No <code>corpus/synthesis_scripts</code> rerun	intentional	The build-kit pipeline can regenerate artifacts from <code>source_materials/</code> .	The current <code>app/artifacts/</code> files are pre-generated and committed.	The artifacts are stable — no need to rerun the synthesis pipeline for this demo. Re-running would require redoing Stage 1 extraction (89 LLM calls) and Stage 3 synthesis.

7. Bugs, blockers, and known issues

Severity	What	Details	Workaround
----------	------	---------	------------

smell	Streamlit caches imported modules across reloads	<code>streamlit run dashboard.py</code> auto-reloads <code>dashboard.py</code> on save but not <code>cj_chat.py</code> (an imported module). Tuning changes in <code>cj_chat.py</code> (TTS pause length, foreign substitutions, etc.) require a full Streamlit restart (Ctrl+C + relaunch).	Document the restart requirement (already in <code>PROJECT.md</code> and <code>app/README.md</code>).
smell	Whisper "medium" model is slow on CPU	3-5s per transcription warm. Build-kit target is ≤ 4 s end-to-end turn, which is unreachable on this hardware with <code>medium</code> .	Switch <code>WHISPER_MODEL=small</code> in <code>.env</code> for English-only demos (drops transcription to ~1s); accept slower turns for Tagalog code-switch.
smell	Piper RTF ~1.8 on CPU	A 150-word response takes ~9s to render.	Acceptable for the demo. Build-kit explicitly suggests OpenAI TTS <code>onyx</code> swap for production polish (~5x faster + better diction). User declined this swap.
tech debt	No tests	Manual smoke-test only. Refactors are risky.	Add minimal pytest suite for <code>_prepare_tts_text</code> , <code>_strip_stage_directions</code> , <code>route_question</code> JSON parsing fallback. ~1-2 hours. See §9.
tech debt	<code>cj_chat.py</code> is 616 lines of one-file mixed concerns	Config + pipeline functions + audio I/O + main loop. Refactoring into modules would help future-CC, but the build kit explicitly intended this to be a "runnable skeleton" single file.	Leave alone for now.
tech debt	<code>synthesize.py</code> and other <code>corpus/synthesis_scripts/</code> are not exercised	The current <code>app/artifacts/</code> are pre-built; the synthesis scripts in <code>corpus/synthesis_scripts/</code> haven't been re-run during this session. If they have bit-rot, no one would know yet.	Document as a "do this before Pass B" task.
smell	TTS phonetic substitutions are crude approximations	Tagalog/Spanish/French speakers will hear the attempt is American-English. Native quality requires a multilingual TTS engine.	Documented in <code>PROJECT.md</code> . User explicitly chose this over OpenAI TTS swap.

smell	<code>record_until_silence</code> uses RMS threshold (350)	Hardcoded ambient-noise threshold. Will fail in loud rooms or with very quiet speakers.	Acceptable for controlled demo environment. Bump threshold or add auto-calibration if needed.
smell	First-time whisper download is ~22 minutes for <code>medium</code>	1.5 GB at apparent ~1 MB/s on the test machine. User experience is "app hangs" if they didn't pre-fetch.	Documented in <code>app/README.md</code> : "(Optional) Pre-fetch the whisper model" with the exact one-liner.
smell	Conversation history in dashboard lives in browser session only	Refresh = lost context. No durable store.	Out of scope per build kit ("No memory beyond last 10 turns").
tech debt	The <code>state/current.json</code> references in PROJECT.md are stale	The old read-only dashboard wrote to <code>state/current.json</code> ; the new interactive dashboard doesn't. PROJECT.md mentions it once.	Minor doc inconsistency. Fix when convenient.

No `blocker` or `bug` (wrong behavior) found.

8. Dependencies and environment

System

- **OS:** Windows (tested on Windows with bash from Git for Windows)
- **Python:** 3.11.9 (CLI confirms `Python 3.11.9`). Repo also has 3.14 and 3.12 installed but neither used.
- **Disk:** ~620 MB in `app/` (without whisper cache); add 1.5 GB for `whisper-medium` cache at `%USERPROFILE%.cache\huggingface\hub\`.
- **No ffmpeg, portaudio, or other native libs required** — `sounddevice` ships its own portaudio, `faster-whisper` ships ctranslate2 wheels, Piper is a single-file Windows exe.

Python dependencies (pinned versions installed today)

```
anthropic==0.102.0
ctranslate2==4.7.1
faster-whisper==1.2.1
numpy==2.4.4
onnxruntime==1.26.0
python-dotenv==1.2.2
scipy==1.17.1
sounddevice==0.5.5
streamlit==1.57.0
tokenizers==0.23.1
```

(Full `pip freeze` available; `requirements.txt` uses `>=` constraints.)

Required environment variables (names only)

- `ANTHROPIC_API_KEY` — Claude API key (must start `sk-ant-...`)
- `WHISPER_MODEL` — `small` or `medium` (default `medium`)

- `PIPER_BIN` — path to Piper executable (default `./piper/piper.exe`)
- `PIPER_VOICE` — path to ONNX voice (default `./voices/en_US-ryan-high.onnx`)

Loaded from `app/.env` via `python-dotenv` with `override=True`. `.env.example` is committed; `.env` is gitignored.

Local-only assets (not committed; user must install)

Asset	Path	Size	Source
Piper TTS binary + dlls	<code>app/piper/</code>	38 MB	https://github.com/rhasspy/piper/releases
Piper voice model (ONNX)	<code>app/voices/en_US-ryan-high</code>	116 MB	https://huggingface.co/rhasspy/piper-voice
Piper voice config	<code>app/voices/en_US-ryan-high</code>	4 KB	<code>onnx.json</code> https://huggingface.co/rhasspy/piper-voice
faster-whisper <code>medium</code> model	<code>%USERPROFILE%\.cache\huggingface\hub\models--Systran\auto-download-onnx-<code>medium</code>\</code>	1.5 GB	auto-downloads on first run from HuggingFace
Anthropic Python SDK	<code>venv</code>	—	pip-installed

`.venv/` is 464 MB; not committed. Rebuild with `py -3.11 -m venv .venv && .venv/Scripts/python.exe -m pip install -r requirements.txt`.

9. Immediate next actions

Ranked by what unblocks the next obvious step.

1. **Add a minimal pytest suite.** [2-4 hours]
Touches: new `app/tests/` directory.
3. Why: every refactor right now is a step into a swamp. A few unit tests on `_prepare_tts_text` (input -> expected chunks), `_strip_stage_directions` (real example + 3 edge cases), `route_question` JSON parsing fallback (malformed router output -> safe default), and a mocked end-to-end `run_turn` would let future-CC change things without fear.

Blocker: none.

Reconcile PROJECT.md with the current dashboard architecture. [30 min]
6. Touches: `PROJECT.md`.
7. Why: the doc was written before the dashboard rewrite (commit `b7072ef`); some references to the old read-only state-file design remain (search for `state/current.json`).

Blocker: none.

Bump Whisper VAD silence to 1000ms. [5 min]
10. Touches: `app/cj_chat.py:141` (`min_silence_duration_ms=500` -> `1000`).
11. Why: build kit explicitly recommends this if hallucinations occur; preemptive insurance for the demo.

Blocker: none.

Wire a sidebar "WHISPER_MODEL" toggle. [15 min]
14. Touches: `app/dashboard.py`.

15. Why: lets the operator switch `small/medium` during a demo without restarting. Useful if the audience is pure-English and you want faster turns.

Blocker: would need to call `get_whisper.clear()` to drop the cached `WhisperModel` on switch.

Document the Streamlit-restart requirement more loudly. [10 min]

18. Touches: `app/README.md` quick-start section.

19. Why: every time the user tuned `cj_chat.py` they hit the stale-import issue. A **Restart Streamlit after editing `cj_chat.py`** callout would have saved several round-trips.

Blocker: none.

Add a sample wav for offline TTS regression testing. [30 min]

22. Touches: `app/tests/fixtures/`.

23. Why: confidence that TTS-pause behavior didn't regress.

Blocker: none.

Voice-card iteration on stage directions. [30 min]

26. Touches: `app/artifacts/voice_card.md`.

27. Why: we strip stage directions post-hoc, but adding an explicit prohibition to the system prompt would avoid wasting tokens on generating them in the first place. Belt-and-suspenders.

Blocker: would change a build-kit artifact (consider whether that's in scope).

Investigate Reachy Mini integration scaffold. [4-8 hours]

30. Touches: new `app/robot/` directory.

31. Why: build kit lists this as out of scope for May 30, but the wider Reachy Mini project lives in this same tree (note the `reachy-mini-dances-library` and `reachy-mini-emotions-library` HuggingFace caches on the machine). Bridging audio I/O to the robot would be the next major arc.

Blocker: need user direction on whether to start this before or after the demo.

Speech corpus ingestion (Pass B). [2-3 days]

34. Touches: new directory under `source_materials/speeches/`; rerun `corpus/synthesis_scripts/`.

35. Why: build kit anticipates ~150 CJ speeches as Pass B. Codebase supports them without changes (`synthesize.py` regenerates `topic_map.json` etc.).

Blocker: requires speech sources + LLM cost for Stage 1 extraction.

Add CI on push. [1 hour]

- Touches: new `.github/workflows/ci.yml`.
- Why: at least lint + the pytest suite (#1) on every push to `main`.
- Blocker: #1.

10. Questions for the human

Strategy-doc vs implementation divergence on dashboard role. The strategy/decision handover (which I haven't seen) may describe the dashboard as a read-only audience viewer (the original commit `8311558` design). The current implementation is a self-contained interactive chat (`b7072ef` onward) because you asked for the mic input to be in the dashboard. Confirm which side wins if these conflict.

Voice-card modifications. Are you OK with me editing `app/artifacts/voice_card.md` (and the upstream `corpus/build_kit/voice_card.md`)? Concretely: adding "do not include stage directions or italicized narration" as an explicit instruction. The build kit treats `voice_card.md` as the canonical specification; modifying it changes contract.

WHISPER_MODEL default for the live demo. Build kit says `medium` for Filipino/English code-switching; I have it set to `medium`. But that costs ~5s/turn extra and the demo may be predominantly English. Want me to switch the default to `small` and have a sidebar toggle, or keep `medium` for safety?

TTS_FOREIGN_SUBSTITUTIONS expansion. The default list has 12 entries covering CJ's most common Tagalog/Spanish/French phrases. The `signature_library.json` has 684 phrases total — many are English but a non-trivial chunk are Tagalog. Want me to scan and pre-populate substitutions for every foreign phrase in the corpus, or wait until they come up in practice?

Reachy Mini scope. The HuggingFace cache on this machine shows you've already pulled `pollen-robotics/reachy-mini-dances-library` and `reachy-mini-emotions-library`. Is the May 30 conversation app meant to feed into the robot at some point, or are these tracks fully independent?

Anthropic API key in `.env` was shared in chat. I noted twice that it should be rotated. Has it been? If not, treat the key string as compromised and rotate via <https://console.anthropic.com/settings/keys>. The local `.env` is gitignored and was never pushed, but the value is in this conversation's history.

The corpus synthesis scripts haven't been re-run during this session. `corpus/synthesis_scripts/synthesize.py`, `make_summary.py`, `taxonomy.py`, `aliases.py` are committed and presumably ran cleanly to produce the current `app/artifacts/`. Are they still expected to work as Pass B preparation, or has anything changed in `corpus/` that might invalidate them? Worth a smoke run before Pass B.

Speech corpus location. The build kit anticipates ~150 speeches from `cjpanganiban.com` — none are in `source_materials/` today. Do you have them collected somewhere (and we just haven't ingested), or is collection itself the next task?

11. How to use this handover

This document is **implementation truth as of 2026-05-15**. It describes what actually exists on disk, what actually runs, and what I (Claude Code) actually did during the build session.

It is the companion to a separate strategic/decision handover written in another Claude conversation. Where the two disagree:

- For **what exists, what runs, what's installed where** -> trust this document and the git tree.
- For **why we made an architectural choice, what the design intent was, what's still open as a design question** -> trust the strategic handover.

When you (future-me or future-Claude) open this project again:

1. Read this document first. Then read `PROJECT.md` for runtime-tuning detail. Then read the strategic/decision handover for design intent.
2. Before assuming anything is true, run `git log --oneline -10` to see what's happened since `839a4d2`, and re-run the smoke test in §4 to verify the pipeline still works.
3. Any code referenced here by file:line is at the commit `839a4d2`. Subsequent commits may have moved things.

4. The 8 questions in §10 are the most likely places where ambiguity bites. Reconcile against this document before acting on the strategic handover's answers.

— Claude Code, 2026-05-15